

随机数

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
int get(int L, int R) {
    return rng() % (R - L + 1) + L;
}
```

二分图最大权匹配

这种方法适用于边权匹配，复杂度为 $O(n^3)$ ，使用邻接矩阵存图，传入左部和右部的大小

```
#include <bits/stdc++.h>
using namespace std;

template<class T>
struct MaxAssignment {
public:
    T solve(int nx, int ny, vector<vector<T>> a) {
        assert(0 <= nx && nx <= ny);
        assert(int(a.size()) == nx);
        for(int i = 0; i < nx; i++) {
            assert(int(a[i].size()) == ny);
            for(auto x : a[i])
                assert(x >= 0);
        }
        auto update = [&](int x) {
            for(int y = 0; y < ny; y++) {
                if(lx[x] + ly[y] - a[x][y] < slack[y]) {
                    slack[y] = lx[x] + ly[y] - a[x][y];
                    slackx[y] = x;
                }
            }
        };

        costs.resize(nx + 1);
        costs[0] = 0;
        lx.assign(nx, numeric_limits<T>::max());
        ly.assign(ny, 0);
        xy.assign(nx, -1);
        yx.assign(ny, -1);
        slackx.resize(ny);
        for(int cur = 0; cur < nx; cur++) {
            queue<int> que;
            visx.assign(nx, false);
            visy.assign(ny, false);
            slack.assign(ny, numeric_limits<T>::max());
            p.assign(nx, -1);
```

```

for(int x = 0; x < nx; x++) {
    if(xy[x] == -1) {
        que.push(x);
        visx[x] = true;
        update(x);
    }
}

int ex, ey;
bool found = false;
while(!found) {
    while(!que.empty() && !found) {
        auto x = que.front();
        que.pop();
        for(int y = 0; y < ny; y++) {
            if(a[x][y] == lx[x] + ly[y] && !visy[y]) {
                if(yx[y] == -1) {
                    ex = x;
                    ey = y;
                    found = true;
                    break;
                }
                que.push(yx[y]);
                p[yx[y]] = x;
                visy[y] = visx[yx[y]] = true;
                update(yx[y]);
            }
        }
    }
    if(found)
        break;
    T delta = numeric_limits<T>::max();
    for(int y = 0; y < ny; y++)
        if(!visy[y])
            delta = min(delta, slack[y]);
    for(int x = 0; x < nx; x++)
        if(visx[x])
            lx[x] -= delta;
    for(int y = 0; y < ny; y++) {
        if(visy[y]) {
            ly[y] += delta;
        } else {
            slack[y] -= delta;
        }
    }
    for(int y = 0; y < ny; y++) {
        if(!visy[y] && slack[y] == 0) {
            if(yx[y] == -1) {
                ex = slackx[y];
                ey = y;
                found = true;
                break;
            }
        }
    }
}

```

```

        que.push(yx[y]);
        p[yx[y]] = slackx[y];
        visy[y] = visx[yx[y]] = true;
        update(yx[y]);
    }
}

costs[cur + 1] = costs[cur];
for(int x = ex, y = ey, ty; x != -1; x = p[x], y = ty) {
    costs[cur + 1] += a[x][y];
    if(xy[x] != -1)
        costs[cur + 1] -= a[x][xy[x]];
    ty = xy[x];
    xy[x] = y;
    yx[y] = x;
}
}
return costs[nx];
}
vector<int> assignment() {
    return xy;
}
pair<vector<T>, vector<T>> labels() {
    return make_pair(lx, ly);
}
vector<T> weights() {
    return costs;
}
private:
    vector<T> lx, ly, slack, costs;
    vector<int> xy, yx, p, slackx;
    vector<bool> visx, visy;
};

```

二分图最大点权匹配

适用于点权匹配，有时可以将边权转化为点权，进而使用此方法获得 $O(n \log n) \rightarrow O(nm)$ 的优秀复杂度。注意邻接表下标从0开始，修改排序函数可以求最小点权匹配

```

class MaxAssignment {
public:
    int solve(int n, int m, vector<vector<int>> &adj, vector<int> &val) {
        assert(adj.size() == n);
        assert(val.size() == n);
        vector<int> yx(m, -1);
        vector<bool> vis(n, false);
        vector<int> p(n);
        iota(p.begin(), p.end(), 0);
        sort(p.begin(), p.end(),
            [&](auto x, auto y) {

```

```

        return val[x] > val[y];
    });
    auto find = [&](auto &&self, int x) -> bool {
        vis[x] = true;
        for(int y : adj[x]) {
            if(yx[y] == -1 || !vis[yx[y]] && self(self, yx[y])) {
                yx[y] = x;
                return true;
            }
        }
        return false;
    };
    int ans = 0;
    for(auto i : p) {
        if(find(find, i)) {
            vis.assign(n, false);
            ans += val[i];
        }
    }
    return ans;
};

```

最小费用流

注意下标从0开始

```

template<class T>
struct MinCostFlow {
    struct _Edge {
        int to;
        T cap;
        T cost;
        _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
    };
    int n;
    vector<_Edge> e;
    vector<vector<int>> g;
    vector<T> h, dis;
    vector<int> pre;
    bool dijkstra(int s, int t) {
        dis.assign(n, numeric_limits<T>::max());
        pre.assign(n, -1);
        priority_queue<pair<T,int>, vector<pair<T,int>>, greater<pair<T,int>>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while(!que.empty()) {
            T d = que.top().first;
            int u = que.top().second;
            que.pop();
            if(dis[u] != d) {
                continue;
            }

```

```

    }
    for(int i : g[u]) {
        int v = e[i].to;
        T cap = e[i].cap;
        T cost = e[i].cost;
        if(cap > 0 && dis[v] > d + h[u] - h[v] + cost) {
            dis[v] = d + h[u] - h[v] + cost;
            pre[v] = i;
            que.emplace(dis[v], v);
        }
    }
}

return dis[t] != numeric_limits<T>::max();
}

MinCostFlow() {}
MinCostFlow(int n_) {
    init(n_);
}

void init(int n_) {
    n = n_;
    e.clear();
    g.assign(n, {});
}

void addEdge(int u, int v, T cap, T cost) {
    g[u].emplace_back(e.size());
    e.emplace_back(v, cap, cost);
    g[v].emplace_back(e.size());
    e.emplace_back(u, 0, -cost);
}

pair<T, T> flow(int s, int t) {
    T flow = 0;
    T cost = 0;
    h.assign(n, 0);
    while(dijkstra(s, t)) {
        for(int i = 0; i < n; i++) {
            h[i] += dis[i];
        }
        T aug = numeric_limits<int>::max();
        for(int i = t; i != s; i = e[pre[i] ^ 1].to) {
            aug = min(aug, e[pre[i]].cap);
        }
        for(int i = t; i != s; i = e[pre[i] ^ 1].to) {
            e[pre[i]].cap -= aug;
            e[pre[i] ^ 1].cap += aug;
        }
        flow += aug;
        cost += aug * h[t];
    }
    return make_pair(flow, cost);
}

struct Edge {
    int from;
    int to;

```

```

    T cap;
    T cost;
    T flow;
};

vector<Edge> edges() {
    vector<Edge> a;
    for(int i = 0; i < e.size(); i += 2) {
        Edge x;
        x.from = e[i + 1].to;
        x.to = e[i].to;
        x.cap = e[i].cap + e[i + 1].cap;
        x.cost = e[i].cost;
        x.flow = e[i + 1].cap;
        a.push_back(x);
    }
    return a;
}
};

```

SPFA

```

constexpr ll inf = 1e18;
struct edge {
    int v, w;
};

void solve() {
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<edge>> adj(n + 1);
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back(edge(v, w));
    }
    vector<ll> d(n + 1, inf), cnt(n + 1, 0);
    vector<bool> vis(n + 1);
    queue<int> q;
    d[s] = 0; vis[s] = 1; q.push(s);
    bool ne_circle = false;
    while(!q.empty()) {
        int u = q.front(); q.pop(); vis[u] = false;
        for(int i = 0; i < adj[u].size(); i++) {
            int v = adj[u][i].v, w = adj[u][i].w;
            if(d[v] > d[u] + w) {
                d[v] = d[u] + w;
                cnt[v] = cnt[u] + 1;
                if(cnt[v] >= n) {
                    ne_circle = true;
                    break;
                }
            }
            if(!vis[v]) q.push(v), vis[v] = true;
        }
    }
}

```

```

    }
    }
    if(ne_circle) break;
}
if(ne_circle) {
    cout << -1 << "\n";
}
for(int i = 1; i <= n; i++) {
    cout << d[i] << " \n"[i == n];
}
}

```

求无向图的欧拉序

```

struct Edge {
    int to, eid;
    friend bool operator < (const Edge &a, const Edge &b) {
        return a.to < b.to;
    }
};

void solve() {
    int n, m, idx = 0;
    cin >> n >> m;
    vector<vector<Edge>> adj(n + 1);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back({v, idx});
        adj[v].push_back({u, idx++});
    }
    int st = 0, oddDegCnt = 0;
    for(int i = n; i >= 1; i--) {
        int deg = adj[i].size();
        if(deg & 1) {
            st = i;
            oddDegCnt++;
        } else if(oddDegCnt == 0) {
            st = i;
        }
    }
    if(oddDegCnt > 2) {
        cout << "No\n";
        return ;
    }
    for(int i = 1; i <= n; i++) {
        sort(adj[i].begin(), adj[i].end());
    }
    vector<int> path;
    path.reserve(m + 1);
    vector<bool> vis(m);

```

```

auto dfs = [&](this auto &&self, int u) -> void {
    auto &cur = adj[u];
    auto it = cur.begin();
    while(it != cur.end()) {
        Edge e = *it;
        it = cur.erase(it);
        int i = e.eid;
        if(vis[i]) continue;
        vis[i] = true;
        int v = e.to;
        self(v);
    }
    path.push_back(u);
};
dfs(st);
reverse(path.begin(), path.end());
cout << "Yes\n";
for(int i = 0; i < m + 1; i++) {
    cout << path[i] << " \n"[i == m];
}
}

```

无向完全图的欧拉路径

```

void solve() {
    int n; cin >> n;
    vector<int> g(n, 0);
    vector vis(n, vector<bool>(n, false));
    if(n % 2 == 0) {
        //去掉n/2-1条边，保证至多两个奇点
        //如果要求欧拉回路的话，i从1开始
        for(int i = 2; i < n; i += 2) {
            vis[i - 1][i] = true;
            vis[i][i - 1] = true;
        }
    }
    //如果不允许自环，加上下面这段for循环
    for(int i = 0; i < n; i++) {
        vis[i][i] = true;
    }
    vector<int> path;
    auto dfs = [&](this auto &&self, int u) -> void {
        for(; g[u] < n; g[u]++) {
            int v = g[u];
            if(!vis[u][v]) {
                vis[u][v] = true;
                vis[v][u] = true;
                self(v);
            }
        }
        path.push_back(u);
    };
}

```



```

dfs(0);
reverse(path.begin(), path.end());
for(int i = 0; i < path.size(); i++) {
    cout << path[i] << " \n"[i == path.size() - 1];
}
}

```

区间内线段数量

树状数组

题意:

给定N条线段[L,R], 有Q次询问

每次询问给定l,r, 分别回答区间[l,r]内两种覆盖的线段数量

(1)允许不完全覆盖数量

(2)完全覆盖数量

$N \in [1, 1e5]$, $L, R \in [1, 1e5]$, $Q \in [1, 1e5]$

```

int n;
vector<int> t1(100005), t2(100005);
void add(vector<int> &a, int i) {
    for(; i <= 100000; i += i & -i) a[i]++;
}
int query(vector<int> &a, int i) {
    int res = 0;
    for(; i > 0; i -= i & -i) res += a[i];
    return res;
}
struct info {
    int l, r;
};
struct Q {
    int l, r, id;
};
void solve() {
    int q;
    cin >> n >> q;
    vector<info> seg(n + 1);
    for(int i = 1; i <= n; i++) {
        int l, r;
        cin >> l >> r;
        seg[i] = {l, r};
    }
    vector<Q> que(q + 1);
    for(int i = 1; i <= q; i++) {
        int l, r;
        cin >> l >> r;
        que[i] = {l, r, i};
    }
    sort(que.begin() + 1, que.end(), [](auto &a, auto &b) -> bool {
        return a.r < b.r;
    });
}

```

```

sort(seg.begin() + 1, seg.end(), [](auto &a, auto &b) -> bool {
    return a.r < b.r;
});
vector<int> ans2(q + 1);
int idx = 1;
for(int i = 1; i <= q; i++) {
    auto [l, r, id] = que[i];
    while(idx <= n && seg[idx].r <= r) {
        add(t1, seg[idx].l);
        add(t2, seg[idx].r);
        idx++;
    }
    int ans = query(t2, r) - query(t1, l - 1);
    ans2[id] = ans;
}
for(int i = 1; i <= q; i++) {
    cout << ans2[i] << " \n"[i == q];
}
for(; idx <= n; idx++) {
    add(t1, seg[idx].l);
    add(t2, seg[idx].r);
}
vector<int> ans1(q + 1);
for(int i = 1; i <= q; i++) {
    auto [l, r, id] = que[i];
    int ans = query(t1, r) - query(t2, l - 1);
    ans1[id] = ans;
}
for(int i = 1; i <= q; i++) {
    cout << ans1[i] << " \n"[i == q];
}
}

```

Python模板

```

import io, os, sys
input = lambda: sys.stdin.readline().strip()

import math
inf = math.inf

def I():
    return input()

def II():
    return int(input())

def MII():
    return map(int, input().split())

```

```
def LI():
    return input().split()

def LII():
    return list(map(int, input().split()))

def LFI():
    return list(map(float, input().split()))

def GMI():
    return map(lambda x: int(x) - 1, input().split())

def LGMI():
    return list(map(lambda x: int(x) - 1, input().split()))

from io import BytesIO, IOBase

import random
import os

import bisect
import typing
from collections import Counter, defaultdict, deque
from copy import deepcopy
from functools import cmp_to_key, lru_cache, reduce
from heapq import merge, heapify, heappop, heappush, heappushpop, nlargest, nsmallest
from itertools import accumulate, combinations, permutations, count, product
from operator import add, iand, ior, itemgetter, mul, xor
from string import ascii_lowercase, ascii_uppercase, ascii_letters
from sortedcontainers import SortedList
from typing import *
BUFSIZE = 4096
```